AI-NATIVE VIDEO CREATION & EDITING PLATFORM

Development Plan

Version 1.0 • Proposed implementation roadmap • 04 November 2026

Objective: Convert the approved functional specification, database schema blueprint and API contract into an executable, testable development roadmap for the MVP. The plan sequences infrastructure, upload and analysis, AI planning, rendering, editor experience, and production readiness.

Technology baseline: React, Node.js/Express, MongoDB 7 + Mongoose 8, FFmpeg, BullMQ/Redis, AWS S3, and Gemini/OpenAI provider APIs.

1. Executive overview

The product lets a user upload a video, describe desired edits in natural language, inspect a generated preview, iterate through prompts or manual edits, and export a final video. Development should proceed in vertical slices: first establish secure project and media foundations; then analysis and captions; then deterministic editing and rendering; then AI-generated plans; finally the complete interactive editor and production hardening.

The source functional specification defines the MVP as upload, analysis, prompt-to-plan, proxy preview, final export and iterative chat edits. Real-time collaboration, mobile editing and custom model training are explicitly out of scope.

2. Product scope and release boundary

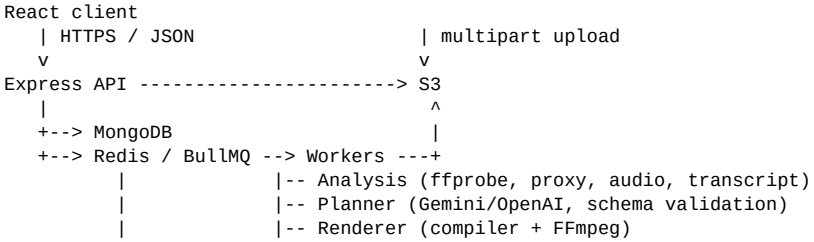
Included in MVP

- User authentication and project management.
- Video upload to S3 using multipart presigned URLs.
- Asset validation, proxy generation, audio extraction and media analysis.
- Transcript and word-timed subtitles.
- Natural-language prompt to validated, versioned Edit Plan JSON.
- Color presets, silence cuts, subtitles, transitions, intro/outro, music and audio processing as staged capabilities.
- 480p preview, final MP4 export, progress reporting, download.
- Iterative prompt edits, manual plan patches, undo/redo and saved history.

Out of scope for MVP

- Real-time multi-user collaboration.
- Native mobile editing application.
- Custom foundation/model training.
- Multi-clip source projects unless explicitly approved as a later schema/product extension.

3. Architecture and delivery principles



```
|
+--> Redis progress --> API SSE --> React
```

Non-negotiable engineering principles

1. LLM output is structured data only; deterministic server code builds FFmpeg argument arrays.
2. Original uploads are immutable. Every accepted edit is versioned and reversible.
3. Preview uses the 480p proxy; final export uses the original asset; both use the same compiler.
4. Workers receive identifiers, load validated records from MongoDB, and run outside API request lifetimes.
5. Every project/asset/plan/job read and write is tenant-scoped.
6. The Edit Plan is the single source of truth for derived editor state.

4. Proposed development schedule

The following is a proposed 10-week MVP schedule for a small team with parallel frontend/backend/worker work. It is a planning estimate, not a duration specified in the source documents. Phase exit criteria are the gates for moving forward; if a gate fails, stabilize that phase before adding more features.

Week	Phase	Primary outcome
1	Foundation & architecture	Repository, environments, auth, project skeleton, MongoDB and S3/Redis connectivity.
2	Project & upload	Project CRUD, multipart upload, S3 completion, asset records and validation.
3	Media analysis	ffprobe, proxy, WAV, transcription, silence/scene/beat analysis and auto-captions.
4	FFmpeg render foundation	Compiler skeleton, preview render, render job state, progress events and output storage.
5	AI planner	Provider adapter, context builder, Edit Plan schema, validation and version persistence.
6	Editing features	Silence cuts, color presets, subtitles, transitions and intro/outro rendering.
7	Frontend editor	Preview player, timeline/overlays, prompt chat, plan sync, patch editing and SSE.
8	Audio & iteration	Music selection, beat sync, ducking, undo/redo, preview regeneration and export flow.
9	Integration & QA	End-to-end tests, performance tuning, security, retries, cleanup and observability.
10	Release readiness	Acceptance suite, bug fixes, deployment, runbooks and MVP sign-off.

5. Detailed workstreams and phase deliverables

Phase 0 — Project foundation (Week 1)

Tasks

• Create frontend, API and worker repositories/packages; define linting, formatting, environment configuration and CI checks.
• Set up local development and staging environments for MongoDB, Redis, S3-compatible storage and FFmpeg.
• Implement Express application structure, request IDs, centralized errors, authentication middleware and authorization helpers.
• Create initial Mongoose models and indexes for users, projects, assets and renderJobs; establish migration/index deployment process.
• Create S3 bucket/prefix conventions and Redis/BullMQ connectivity.

Exit criteria: A developer can authenticate, create a project, persist it in MongoDB, and run API/worker processes against configured services.

Phase 1 — Project management and video upload (Week 2)

Tasks

• Implement project create/list/read/update/archive endpoints and React project dashboard.
• Implement POST /uploads to create multipart S3 sessions and return short-lived part URLs.
• Implement browser-side chunk upload, retry of failed parts, progress display and completion request.
• Implement POST /uploads/:id/complete, verify parts, finalize multipart upload, create Asset record and enqueue analysis.
• Enforce project ownership, file size/type limits, checksum handling, abandoned-upload cleanup and ffprobe validation.

Exit criteria: A user can upload a supported video, see its asset status, and the server creates one analysis job. Upload is direct to S3; Express does not proxy large file bytes.

Phase 2 — Media analysis and captions (Week 3)

Tasks

• Build analysis worker to download/read raw asset, run ffprobe and record duration, dimensions, fps, codecs and audio metadata.
• Generate 480p proxy and 16 kHz mono WAV; store outputs under the documented per-user/project S3 prefixes.

• Integrate transcription (Whisper API or self-hosted faster-whisper decision remains open) with word timestamps.
• Detect silence ranges, scene cuts, beat grid and loudness; persist analysisResults separately from large media assets.
• Generate caption sets and a first captioned preview using the renderer foundation or a controlled interim pipeline.
• Expose analysis state and summary to the project page.

Exit criteria: A five-minute test video produces a captioned preview with word timing within 150 ms, matching the source specification's Phase 1 acceptance criterion.

Phase 3 — Render engine foundation (Week 4)

Tasks

• Define RenderJob lifecycle and BullMQ queues for analysis, preview and render with configured concurrency, timeout and retry policies.
• Implement pure compiler interface compile(plan, analysis, profile) -> args/inputs/subtitle files.
• Run FFmpeg with child_process.spawn and an argument array; do not use shell command interpolation.
• Create isolated per-job temporary directories and guaranteed cleanup.
• Parse FFmpeg -progress output, calculate percent, publish Redis job:{id} events and relay via GET /jobs/:id/events SSE.
• Upload preview output to S3 and issue expiring download URLs.

Exit criteria: A known static edit plan can render a valid 480p preview; progress reaches the client; job state is durable and a failed job leaves the source and plan intact.

Phase 4 — AI planner and versioned plans (Week 5)

Tasks

• Implement PlannerProvider adapter for Gemini and/or OpenAI; choose the initial production provider/model in configuration.
• Build context builder using prompt, current plan, compact analysis summary and allowed catalog data; do not send raw video.
• Define Edit Plan JSON schema and Zod discriminated unions for layer types and parameters.
• Implement structured output/function calling, low-temperature generation, parameter clamping, reference resolution and time sanity checks.
• Implement one schema-repair retry; reject a second invalid response with a friendly error.
• Persist immutable editPlans with projectId, version, parentVersion, prompt, plan and provider/model metadata.
• Implement follow-up patch operations so edits only change requested properties.

Exit criteria: At least 95% of a 50-prompt test set yields valid plans on first try, matching the source specification's Phase 3 acceptance criterion.

Phase 5 — Core editing features (Week 6)

Tasks

• Implement silence cut layers and source-to-output time resolution.
• Implement color presets and numeric controls for brightness, contrast, saturation, LUT, vignette and grain.
• Implement subtitle styling and ASS generation with per-word timing/animation.
• Implement transitions with xfade/acrossfade; implement intro/outro templates and normalize dimensions/fps before concat.
• Add validation for source ranges, frame-grid alignment, duration, max layer count and allowed parameter ranges.
• Build regression clips for audio/video sync and effect correctness.

Exit criteria: Silence cut result has no gaps over 0.6 seconds; color preset visibly changes frames; transitions and intro/outro render without A/V drift above 40 ms.

Phase 6 — Frontend editor experience (Week 7)

Tasks

• Build React editor shell with project panel, preview player, prompt/chat panel, timeline and export dialog.
• Use TanStack Query for server data; Zustand with Immer for editor state and derived selectors.
• Keep Edit Plan as the source of truth; derive timeline tracks, overlays and inspector from plan.layers.
• Implement playback state, timeline zoom/scroll, layer selection, snapping and Canvas 2D overlays.

• Implement prompt submit, plan version replacement, preview stale state and SSE progress display.
• Implement debounced manual edits and PATCH requests with optimistic concurrency handling.

Exit criteria: A user can load a project, inspect its current plan, submit a prompt, see the new plan version, and watch preview progress without losing server state.

Phase 7 — Audio, history and export (Week 8)

Tasks

• Implement music selection from licensed catalog, fade in/out, speech ducking and beat synchronization.
• Implement audio loudness normalization and optional denoise.
• Implement undo/redo through immutable plan/timeline version history, not ad hoc local-only reversals.
• Implement final export profiles (1080p default; 4K only if tier/worker capacity permits).
• Implement render completion callback, idempotency, download URL issuance and project publishedVersion update.
• Complete iterative chat edit flow: follow-up prompt creates patch, new version, fresh preview.

Exit criteria: Cuts snap to beats within 60 ms; speech remains intelligible with at least 8 dB ducking; 'Make subtitles bigger' changes only that parameter; undo restores the previous version.

Phase 8 — Integration, QA and performance (Week 9)

Tasks

• Run end-to-end test paths from registration through upload, analysis, AI prompt, preview, revision, export and download.
• Test invalid/corrupt media, oversized uploads, expired URLs, missing catalog references, provider rate limits and malformed model output.
• Test duplicate/out-of-order worker callbacks, queue retries, worker restart and SSE reconnect recovery.
• Test optimistic concurrency conflicts and tenant isolation across projects, assets, plans, jobs and download URLs.
• Measure API availability, analysis speed, planner p95, preview render speed, scrubbing latency and canvas draw time.
• Add structured logs, queue depth/failure alerts, job metrics and retention cleanup.

Exit criteria: Critical paths pass automated tests; no cross-tenant access; error/retry behavior is predictable; performance targets are measured and documented.

Phase 9 — Release and handover (Week 10)

Tasks

• Deploy production API, workers, MongoDB, Redis and S3 configuration with secrets managed outside source control.
• Configure bucket encryption, per-user key prefixes, retention/lifecycle policies and delete-on-request purge.
• Complete smoke tests, backup/restore check, worker capacity check and rollback plan.
• Prepare operations runbook for failed uploads, failed renders, queue backlog, provider outages and S3 issues.
• Review open architectural decisions: intro/outro rendering approach, transcription hosting, GPU encoding and multi-clip project scope.
• Obtain product acceptance sign-off and tag the MVP release.

Exit criteria: Release checklist signed; core acceptance tests pass in production-like environment; support and recovery procedures are documented.

6. Feature priority and dependency map

Priority	Features	Reason / dependency
P0 — Foundation	Auth, project CRUD, storage, upload, asset metadata, queue setup, job status.	Required before any editing workflow can operate safely.
P1 — First usable video	ffprobe, proxy, WAV, transcription, word-timed captions, captioned preview.	Delivers first demonstrable result and validates media pipeline.
P2 — Editing core	Silence cuts, color presets, subtitle styling, preview compiler, export.	Creates meaningful editing value before advanced AI iteration.
P3 — AI-native workflow	Provider adapter, prompt context, schema output, plan validation, versioned edits.	Enables natural-language editing on top of deterministic renderer.
P4 — Rich editing	Transitions, intro/outro, music, beat sync, ducking, manual timeline controls.	Adds polish after core render reliability.
P5 — Production	Undo/redo, security hardening, metrics, retention, load testing, operational runbooks.	Makes the MVP dependable for real users.

7. Database implementation sequence

Implement collections in dependency order and keep unbounded or large data out of parent documents. The database blueprint emphasizes single-read editor sessions, immutable history, separation of AI intent from resolved timeline state, document-size discipline and optimistic concurrency.

```
1. users
2. projects
3. assets
4. analysisResults
5. editPlans
6. timelines (current editable head)
7. timelineVersions (immutable snapshots)
8. captionSets / captionChunks
9. chatMessages
10. renderJobs
11. catalog
```

Required data rules to enforce

• Public IDs use prefixed strings; MongoDB ObjectIds remain internal.
• Timeline time values use integer milliseconds snapped to the frame grid; Edit Plan time values are converted during resolve.
• Every document carries userId and relevant projectId for tenant scoping and indexing.
• Embed bounded data read together (tracks, clips, effects, keyframes); reference large/unbounded data (assets, captions, analysis, jobs).
• Use immutable snapshots for accepted edits and optimistic rev checks for concurrent timeline writes.
• Enforce timeline limits: maximum 12 tracks and 2,000 clips; subtitle cues remain outside the timeline document.

8. Testing and acceptance strategy

Testing should be tied to each phase exit gate, then repeated as an end-to-end regression suite before release.

Test layer	What to verify
Unit	Edit Plan schema validation, clamping, reference resolution, time mapping, compiler argument generation, state transitions, callback signature/idempotency.
Integration	MongoDB persistence/indexes, S3 multipart lifecycle, BullMQ enqueue/retry, Redis progress relay, provider adapter, FFmpeg worker and download URL issuance.
End-to-end	Create project → upload → analyze → prompt → preview → edit prompt → undo/redo → export → download.
Security	Authentication, project ownership, IDOR attempts, signed URL expiry, file sniffing, callback HMAC, prompt rate limits and secret redaction.

Test layer	What to verify
Performance	Planner p95, analysis throughput, preview render speed, SSE reconnect, timeline scrubbing and canvas draw time.
Resilience	Worker restart, duplicate callback, S3 transient failure, provider timeout/rate limit, corrupt input and queue backlog.

9. MVP acceptance criteria

The following measurable criteria are drawn from the functional specification and should be treated as release gates:

• Phase 1: A five-minute video yields a captioned preview with word timing within 150 ms.
• Phase 2: Silence-cut output has no gaps over 0.6 seconds; color preset visibly changes frames.
• Phase 3: At least 95% of a 50-prompt test set yields valid plans on first try.
• Phase 4: Transitions and intro/outro render without A/V desynchronization above 40 ms.
• Phase 5: Cuts snap to beats within 60 ms; speech remains intelligible with ducking of at least 8 dB.
• Phase 6: 'Make subtitles bigger' changes only the requested parameter; undo restores the prior version.
• Non-functional: analysis under 0.5× realtime for a 10-minute video; plan generation under 15 seconds p95; preview render under 0.5× realtime at 480p; scrubbing under 100 ms; canvas draw under 4 ms per frame at 1080p CSS size.
• Operations: 99.5% API availability target; uploads up to 5 GB and 60 minutes (MVP); encrypted S3 buckets, per-user key prefixes and delete-on-request purge.

10. Team responsibilities (suggested)

Roles can be combined in a small team; the split below clarifies ownership rather than requiring a specific headcount.

Role	Primary ownership
Frontend engineer	React project dashboard/editor, upload UX, preview player, timeline, prompt/chat, SSE and export UI.
Backend engineer	Express API, authentication, project/asset/plan/job endpoints, MongoDB schemas/indexes, S3 presigned URLs and provider adapter.
Media/worker engineer	BullMQ workers, ffprobe, transcription/analysis, FFmpeg compiler, render profiles, progress reporting and worker isolation.
QA / automation	Acceptance fixtures, API integration tests, media sync checks, browser E2E, security and regression testing.
Tech lead / product owner	Scope, architecture decisions, schema/API reviews, phase gates, risk management and release sign-off.

11. Risks, open decisions and mitigations

Risk / open decision	Impact	Action
Video processing cost and worker capacity	Long render queues and high infrastructure cost.	Separate preview/render queues; measure real workloads; apply worker limits and per-user quotas.
LLM output variability	Invalid plans or unintended edits.	Structured output, strict schema, clamp/resolve, one repair retry, test prompt set and immutable versions.
Large uploads / unreliable networks	Failed or abandoned uploads.	Multipart S3, resumable part retries, checksum validation, lifecycle abort of abandoned sessions.
A/V timing errors	Unusable output.	Frame-grid time model, shared preview/final compiler, regression media fixtures and drift assertions.
Intro/outro rendering	Implementation choice remains open in source.	Decide Remotion server rendering vs prebuilt Lottie/After Effects clips before Phase 5.
Transcription hosting	Cost/latency and operational complexity.	Choose Whisper API vs self-hosted faster-whisper based on volume, privacy and budget.

Risk / open decision	Impact	Action
GPU encoding	Capacity and cost implications.	Start with CPU worker profile; benchmark before enabling NVENC or 4K tier.
Multi-clip projects	Requires product/schema expansion.	Keep out of MVP unless product priority changes; consider Edit Plan clips[] in v1.1.

12. Definition of done and release checklist

A feature is done only when its acceptance behavior is implemented, API/data contracts are documented, tenant authorization is tested, failures are handled, logs/metrics are available, and regression tests pass.

Release checklist

- ■ All MVP flows work end to end in staging.
- ■ No secrets are present in frontend bundles, source control or logs.
- ■ All object storage is private; signed URLs are short-lived.
- ■ Worker callbacks are authenticated, replay-protected and idempotent.
- ■ Failed jobs preserve source assets and validated plan history.
- ■ Upload and preview cleanup policies are active.
- ■ Backup/restore and operational runbooks are verified.
- ■ Performance and acceptance metrics are recorded.
- ■ Product owner signs off on MVP scope and known limitations.

13. Source alignment and planning assumptions

This roadmap is based on the uploaded AI Video Editing Platform Functional & Technical Specification and Database Schema Blueprint, together with the API Contract & Architecture Design created in this conversation. Feature acceptance thresholds and non-functional targets are carried forward from the functional specification. Database sequencing and data integrity rules follow the schema blueprint.

The 10-week schedule, team roles, priority order and detailed task breakdown are planning recommendations added to make the specifications executable. They are not commitments or durations stated in the source documents. Re-estimate after confirming team size, available infrastructure, provider choice and media-processing budget.

End of document